



Lecture[02]

ARRAYS AND FUNCTIONS



Agenda

- Arrays
 - Declaration
 - Initialization
 - Character array
 - Multiple-Subscripted Arrays



Arrays

- Array
 - Group of consecutive memory locations
 - Same name and type
 - Same size throughout program
- To refer to an element, specify
 - Array name
 - Position number
- Format: *arrayname* [*position number*]
 - First element at position **0**
 - n element array named **c**: **c[0]**, **c[1]**...**c[n-1]**

Name of array (Note that all elements of this array have the same name, **c**)

c[0]	-45
c[1]	6
c[2]	0
c[3]	72
c[4]	1543
c[5]	-89
c[6]	0
c[7]	62
c[8]	-3
c[9]	1
c[10]	6453
c[11]	78

Position number of the element within array **c**



Arrays

- Array elements are like normal variables

```
c[0] = 3;
```

```
printf( "%d", c[0] );
```

- Perform operations in subscript. If $x = 3$,
 $c[5-2] == c[3] == c[x]$



Declaring Arrays

- When declaring arrays, specify

- Name
- Type of array
- Number of elements

```
arrayType arrayName[ numberOfElements ];
```

```
int c[ 10 ];
```

```
float myArray[ 3284 ];
```

- Declaring multiple arrays of same type

- Format similar to regular variables

```
int b[ 100 ], x[ 27 ];
```



Examples Using Arrays

- Initializers

```
int n[5] = {1, 2, 3, 4, 5};
```

- If not enough initializers, rightmost elements become 0
- If too many, syntax error/warning

```
int n[5] = {0}
```

- All elements 0
 - C arrays have no bounds checking
 - If size omitted, initializers determine it
- ```
int n[] = { 1, 2, 3, 4, 5};
```
- 5 initializers, therefore 5 element array



# Validated index

- Never try to exceed size of array

– `int a[100]; a[100]=40;`



Segmentation fault! Uncontrollable behavior

- Compiler of runtime might not check out of bounds
- Validate index `[0, size-1]`
- Bound check, especially input to the array



# Exercises

- Write a program, in which one can input integers from 0 to 9. The program need to count the times of each integer. One can input -1 to stop the program.

Input:

```
0
1
2
5
5
6
7
2
3
4
-1
```

Output:

```
0:1
1:1
2:2
3:1
4:1
5:2
6:1
7:1
8:0
9:0
```





# Character arrays

- Character arrays
  - String "hello" is really a **static** array of characters
  - Character arrays can be initialized using string constants
    - `char string1[] = "first";`
      - null character '`\0`' terminates strings
      - `string1` actually has 6 elements

```
char string1[] = { 'f', 'i', 'r', 's', 't', '\0' };
```



# Character arrays (continued)

- Access individual characters
  - `string1[3]` is character 's'
- Array name is address of array, so `&` not needed for `scanf`
  - `scanf( "%s", string2 ) ;`
  - Reads characters until whitespace encountered
  - Can write beyond end of array, be careful



# Multiple-Subscripted Arrays

- Multiple subscripted arrays
  - Tables with rows and columns ( $m$  by  $n$  array)
  - Like matrices: specify row, then column

|       | Column 0             | Column 1             | Column 2             | Column 3             |
|-------|----------------------|----------------------|----------------------|----------------------|
| Row 0 | <code>a[0][0]</code> | <code>a[0][1]</code> | <code>a[0][2]</code> | <code>a[0][3]</code> |
| Row 1 | <code>a[1][0]</code> | <code>a[1][1]</code> | <code>a[1][2]</code> | <code>a[1][3]</code> |
| Row 2 | <code>a[2][0]</code> | <code>a[2][1]</code> | <code>a[2][2]</code> | <code>a[2][3]</code> |

Diagram illustrating the structure of a multiple-subscripted array (matrix) with annotations:

- Array name:** Points to the `a` in the expression `a[2][1]`.
- Row subscript:** Points to the `2` in the expression `a[2][1]`.
- Column subscript:** Points to the `1` in the expression `a[2][1]`.



# Multiple-Subscripted Arrays

- Initialization

- Initializers grouped by row in braces
- Line up initialization
- If not enough, unspecified elements set to zero
- Only specified the column number

```
int b[2][2] = { { 1, 2 }, { 3, 4 } };
```

|   |   |
|---|---|
| 1 | 2 |
| 3 | 4 |

```
int b[2][2] = { 1,2,3,4 };
```

```
int b[2][2] = { { 1 }, { 3, 4 } };
```

```
char vowels[][5] = {
 {'A', 'E', 'I', 'O', 'U'},
 {'a', 'e', 'i', 'o', 'u'}
};
```

|   |   |
|---|---|
| 1 | 0 |
| 3 | 4 |

- Referencing elements

- Specify row, then column

```
printf("%d", b[0][1]);
```



# Traverse of an array

- Simple array

```
int i = 0;
for (i=0; i<5; i++){
 a[i]=i+1;
}
```

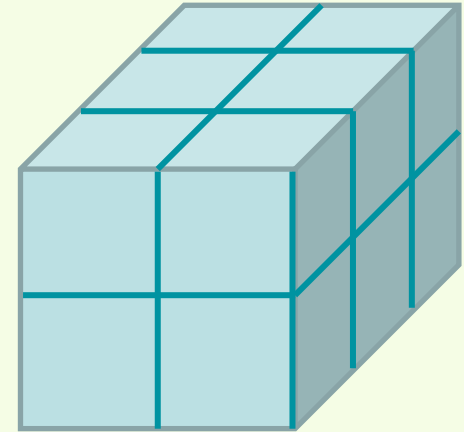
- Multiple-Subscripted Arrays

```
int i = 0, j = 0;
for (i=0; i<2; i++){
 for (j=0; j<2; j++){
 b[i][j]=i+j;
 }
}
```



### Try: 3-dimension array

- Build a 3-d array `a[2][3][2]` and initialized with 1-12;
- Draw the 3-d array and give each element value: such as `a[1][2][1]`
- Three expresses of initialization





# Agenda

- Functions
  - Definition
  - Parameters & Arguments
  - Local variables
  - Call by value
  - Call by reference
  - Passing Arrays to Functions



# Functions

- Functions

- Modularize a program
- All variables declared inside functions are local variables
  - Known only in function defined
- Parameters
  - Communicate information between functions
  - Local variables

```
#include <stdio.h>

int reverse(int a)
{
 int r = 0;
 while(a)
 {
 r = r * 10 + a % 10;
 a = a / 10;
 }
 return r;
}

int main()
{ ...
 if (j + reverse(j) == m)
 ...
}
```





# Functions

- Benefits
  - Divide and conquer
    - Manageable program development
  - Software reusability
    - Use existing functions as building blocks for new programs
    - Abstraction - hide internal details (library functions)
  - Avoids code repetition



# Function Definitions (I)

- Function definition format

*return-value-type function-name ( parameter-list )*

{

*declarations and statements*

}

```
int sum(int a, int b)
```

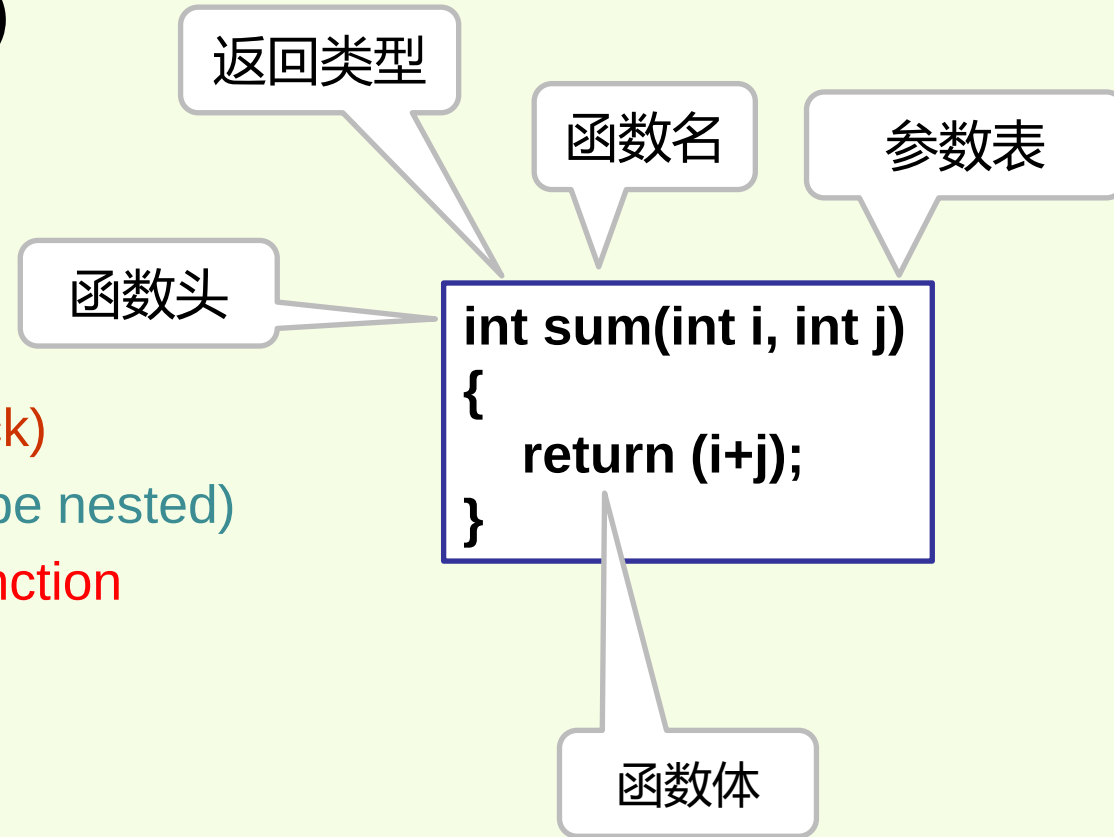
- Function-name: any valid identifier
- Return-value-type: data type of the result (default `int`)
  - `void` - function returns nothing
- Parameter-list: comma separated list, declares parameters (default `int`)



## Function Definitions (II)

```
return-value-type function-name (parameter-list)
{
 declarations and statements
}
```

- Declarations and statements: function body (block)
  - Variables can be declared inside blocks (can be nested)
  - Function can not be defined inside another function
- Returning control
  - If nothing returned
    - **return** ;
    - or, until reaches right brace }
  - If something returned
    - **return** *expression* ;





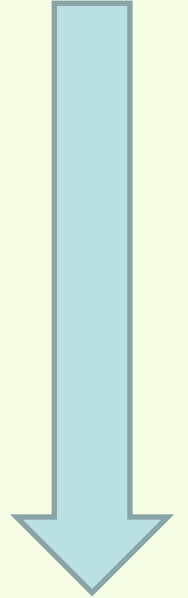
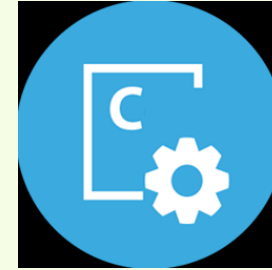
# Function Prototypes (函数原型)

- Function prototype

- Function name
- Parameters - what the function takes in
- Return type - data type function returns (default `int`)
- Used to validate functions
- Prototype only needed if function definition comes after use in program

```
int maximum(int, int, int);
```

- Takes in 3 `ints`
- Returns an `int`





# Function Prototypes

- 函数原型的主要作用和好处

- **实现“分离编译”和模块化**：这是最大的好处。你可以把函数原型放在头文件（.h 文件）里，而把函数的完整实现放在另一个源文件（.c或.cpp文件）里。这样不同的文件可以单独编译，最后再链接在一起。大型项目必须这样做。
- **提高代码可读性和可维护性**：把所有主要的函数原型放在文件开头（或头文件中），就像一本书的目录，让读者一眼就能知道这个程序提供了哪些功能接口，而不需要去翻看每个函数的具体实现细节。
- **让编译器进行类型检查**：这是保证代码安全的重要一环。编译器通过对比函数原型和函数调用，可以检查你是否传入了正确类型和数量的参数。如果传入一个 float 但原型要求是 int，编译器会给出警告，帮助你提前发现潜在的错误。



# Program Modules in C

- Function calls
  - Invoking functions
    - Provide function name and arguments (data)
    - Function performs operations or manipulations
    - Function returns results
  - Boss asks worker to complete task
    - Worker gets information, does task, returns result
    - Information hiding: boss does not know details
- Promotion rules and conversions
  - Converting to lower types can lead to errors

```
void do996(){
 printf("996 is lucky\n");
}
```

```
int main(){
 do996;
 return 0;
}
```



# Program Modules in C

在表达式中混合使用不同类型的数据时，C语言编译器自动进行类型转换的一套规则。目的是在不丢失数据的前提下完成计算，尽可能地保留数据的精度。主要分两种：

- 1、整型提升：在表达式中，凡是 char, short 等小于 int 类型的整型值，在参与运算之前都会被自动转换为 int 或 unsigned int 类型。这个过程是自动的。
- 2、算术转换：当一个运算符（如 +, -, \*, /）的两个操作数是不同的算术类型时，编译器会将它们转换为一个“共同”的类型后再运算。通常是将较低等级的类型转换为较高等级的类型，以防止精度丢失。大致的转换等级（从低到高）： int -> unsigned int -> long -> unsigned long -> long long -> unsigned long long -> float -> double -> long double

注意：把值从较高类型转换到较低类型，会引起数据的丢失。



# Parameters and arguments (形参 vs 实参)

- Inside the function, the values that are passed get assigned to variable called **parameters**

– `void sing(int n)`

- The values that control how the function does its job called arguments (real values)

– `sing(2)`

- Arguments may be constants, variables, or expressions

```
void sing (int n){
 int i = 0;
 for (i = 0;i < n;i++){
 printf("laLALAlala\n");
 }
}
```

```
int main() {
 sing (2);
 return 0;
}
```





# Local Variable

- Once a function is invoked, a memory space is allocated to this function, all variables stored in this space are local variables to this function.
- All variables declared inside the function are local variables.
- Parameters are local variables, too.

```
#include <stdio.h>
void swap (int a, int b){
 int t = a;
 a = b;
 b = t;
}

int main() {
 int i = 3;
 int j = 7;
 swap(i,j);
 return 0;
}
```



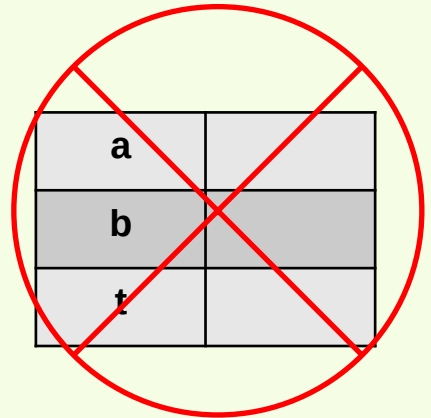
## Rules on local variable

- Local variable are validated inside the function (block within {})
- Variable outside the block is still validated in the block
  - But if a variable with same name is re-defined in the block, the expressions act on the new variable
  - Can't re-define a variable within same block

```
int main() {
 int i = 3;
 int j = 7;
 swap(i,j);
 printf("after swap i=%d, j=%d\n", i, j);
 return 0;
}
```

```
void swap (int a, int b){
 int t=a;
 a = b;
 b = t;
}
```

所以， swap  
了个寂寞？





# Calling Functions: Call by Value

- Used when invoking functions
- Call by value
  - Copy of argument passed to function
  - Changes in function do not effect original
  - Use when function does not need to modify argument
    - Avoids accidental changes

```
void addone(int n) {
 n++;
}
```

```
int i=10;
printf("Before: %d\n", i);
addone(i);
printf("After: %d\n", i);
```



# Call by Reference

- Call by reference (\*)
  - Passes original argument
  - Changes in function effect original
  - Only used with trusted functions

```
void addone(int *n) {
 (*n)++;
}
```

```
int i=10;
printf("Before: %d\n", i);
addone(&i);
printf("After: %d\n", i);
```



# Passing Arrays to Functions

- Passing array elements
  - Passed by call-by-value
  - Pass subscripted name (i.e., `myArray[3]`) to function

```
int myArray[2]={0,1};
```

```
myFunctionVal(myArray[1], 24);
```



# Passing Arrays to Functions

- Passing arrays
  - Specify array name without brackets

```
int myArray[24];
```
  - ```
void myFunctionRef(int[], int length)
```
 - ```
myFunctionRef(myArray, 24);
```
  - Array size usually passed to function
  - Arrays passed call-by-reference
  - Name of array is address of first element
  - Function knows where the array is stored
  - Modifies original memory locations

```

1 /* Fig. 6.13: fig06 13.c
2 Passing arrays and individual array elements to functions
3 #include <stdio.h>
4 #define SIZE 5
5
6 void modifyArray(int [], int); /* appears strange */
7 void modifyElement(int);
8
9 int main()
10 {
11 int a[SIZE] = { 0, 1, 2, 3, 4 }, i;
12
13 printf("Effects of passing entire array call "
14 "by reference:\n\nThe values of the "
15 "original array are:\n");
16
17 for (i = 0; i <= SIZE - 1; i++)
18 printf("%3d", a[i]);
19
20 printf("\n");
21 modifyArray(a, SIZE); /* passed call by reference */
22 printf("The values of the modified array are:\n");
23
24 for (i = 0; i <= SIZE - 1; i++)
25 printf("%3d", a[i]);
26
27 printf("\n\nEffects of passing array element call "
28 "by value:\n\nThe value of a[3] is %d\n", a[3]);
29 modifyElement(a[3]);
30 printf("The value of a[3] is %d\n", a[3]);
31 return 0;
32 }

```

## Function Prototypes

- Function prototypes
- Pass array to a function
- Pass array element to a function
- Print

Entire arrays passed call-by-reference, and can be modified

Array elements passed call-by-value, and cannot be modified

```

33
34 void modifyArray(int b[], int size)
35 {
36 int j;
37
38 for (j = 0; j <= size - 1; j++)
39 b[j] *= 2;
40 }
41
42 void modifyElement(int e)
43 {
44 printf("Value in modifyElement is %d\n", e *= 2);
45 }

```

- Function definitions

Effects of passing entire array call by reference:

The values of the original array are:

0 1 2 3 4

The values of the modified array are:

0 2 4 6 8

Effects of passing array element call by value:

The value of a[3] is 6

Value in modifyElement is 12

The value of a[3] is 6

- Program Output





# Summary

- Array
  - Static length
  - Declaration and initiation
  - $[0, \text{length}-1]$
- Function
  - Parameters & Arguments
  - Local variables
  - Call by value
  - Call by reference



# Recursion

- Recursive functions
  - Function that calls itself
  - Can only solve a base case
  - Divides up problem into
    - What it can do
    - What it cannot do - resembles original problem
      - Launches a new copy of itself (recursion step)
- Eventually base case gets solved
  - Gets plugged in, works its way up and solves whole problem



# Recursion

- Example: factorial:

$$5! = 5 * 4 * 3 * 2 * 1$$

Notice that

$$5! = 5 * 4!$$

$$4! = 4 * 3! \dots$$

- Can compute factorials recursively
- Solve base case ( $1! = 0! = 1$ ) then plug in

- $2! = 2 * 1! = 2 * 1 = 2;$

- $3! = 3 * 2! = 3 * 2 = 6;$



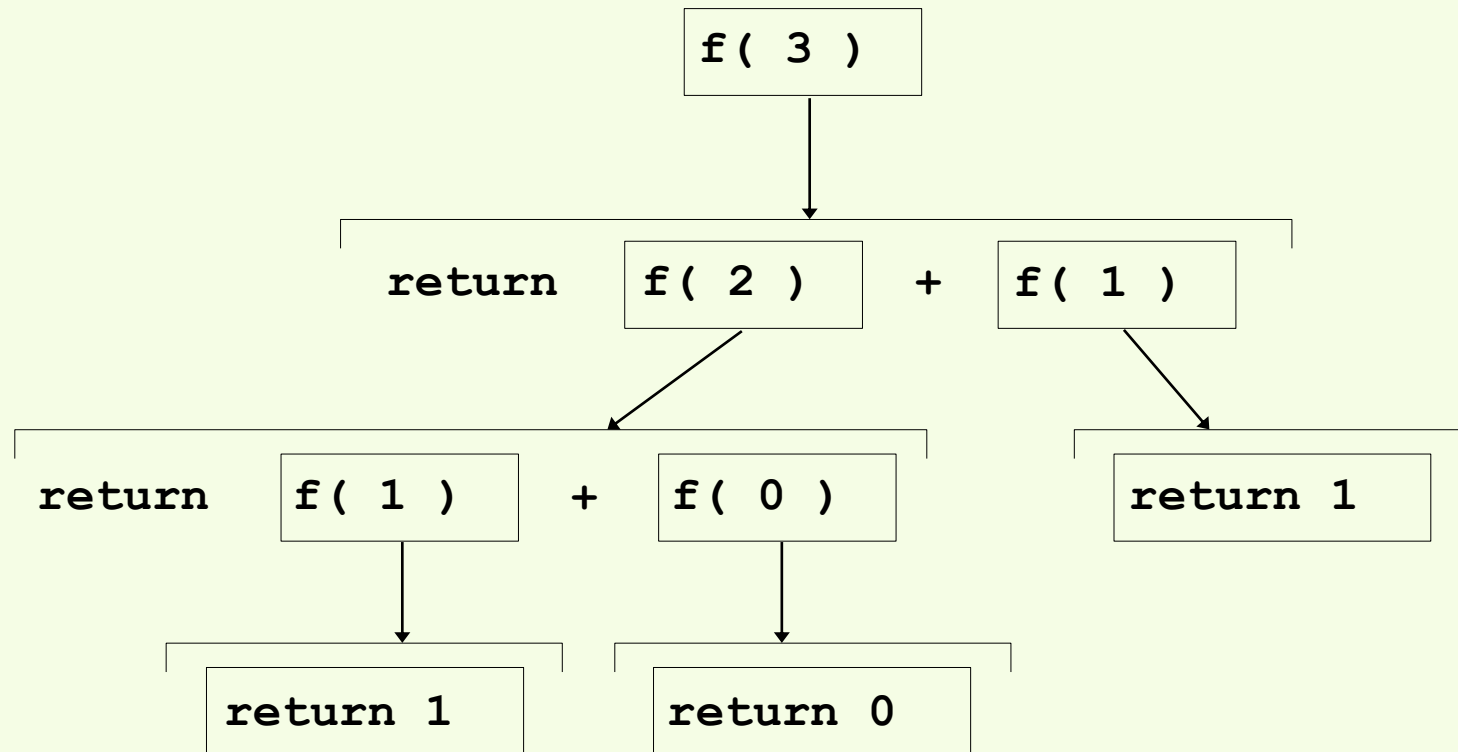
# The Fibonacci Sequence

- Fibonacci series: 0, 1, 1, 2, 3, 5, 8...
  - Each number is the sum of its previous two numbers
$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$$
  - recursive formula

```
long fibonacci(long n)
{
 if (n==0 || n==1) //base case
 return n;
 else
 return fibonacci(n-1) + fibonacci(n-2);
}
```



# The Fibonacci Sequence



```

1 /* Fig. 5.15: fig05_15.c
2 Recursive fibonacci function */
3 #include <stdio.h>
4
5 long fibonacci(long);
6
7 int main()
8 {
9 long result, number;
10
11 printf("Enter an integer: ");
12 scanf("%ld", &number);
13 result = fibonacci(number);
14 printf("Fibonacci(%ld) = %ld\n", number, result);
15 return 0;
16 }
17
18 /* Recursive definition of function fibonacci */
19 long fibonacci(long n)
20 {
21 if (n == 0 || n == 1)
22 return n;
23 else
24 return fibonacci(n - 1) + fibonacci(n - 2);
25 }

```

```

Enter an integer: 0
Fibonacci(0) = 0

```

```

Enter an integer: 1
Fibonacci(1) = 1

```

Enter an integer: 2

Fibonacci(2) = 1

Enter an integer: 3

Fibonacci(3) = 2

Enter an integer: 4

Fibonacci(4) = 3

Enter an integer: 5

Fibonacci(5) = 5

Enter an integer: 6

Fibonacci(6) = 8

Enter an integer: 10

Fibonacci(10) = 55

Enter an integer: 20

Fibonacci(20) = 6765

Enter an integer: 30

Fibonacci(30) = 832040

Enter an integer: 35

Fibonacci(35) = 9227465



# Exercises

- Print out a integer in reverse order
  - 12345 -> 54321
- void reverDigit(int n)
- Recursive way